

Password Router Agent Design Doc

Architecture and tools used

Goal

The router is a password-checking agent. It accepts raw password candidates, runs cheap deterministic checks first, and only escalates to LLM token-probability scoring when the password is not already clearly weak or clearly strong.

High-level architecture

```
User input: one raw password per line
-> Stage 0: deterministic local checks
-> LLM stage 1: Qwen2.5-1.5B token-DP, template A0
-> LLM stage 2: Qwen2.5-1.5B token-DP, template C3
-> Optional escalation: Mistral-7B on the same two templates
-> Verdict: reject, accept, or autoaccept
```

Inputs and outputs

Item	Design choice
Input format	raw password strings, one per line
Forbidden input	no split markers, no annotations, no real passwords currently in use
Primary output	verdict and reason for each password
Audit output	text report and JSON records with stage traces

Trust boundary

The demo is a local/Colab research artifact, not a deployed authentication service. If adapted into a web app, the client-side check should only be UX feedback; the server must rerun the policy before hashing or accepting a password.

Stage 0 Design

Cheap deterministic prechecks before any model is loaded

Purpose

Stage 0 handles the easy cases cheaply. It rejects obvious weak passwords before loading an LLM and provides a reproducible baseline that is easier to explain than neural scoring.

Checks used

- zxcvbn whole-password score with reject cutoff $\log_{10} < 10$.
- Stable substring dynamic program that adds zxcvbn-derived substring candidates.
- Aho-Corasick bad-substring resource from `data/stage0_bad_substrings.csv`.
- Pwned exact and normalized/base variant checks when configured in the demo notebook.
- Patch parse output, so the report can explain why Stage 0 rejected a password.

Important design choice

Stage 0 is intentionally conservative and cheap. It is not meant to fully solve semantic predictability. It prevents the LLM from wasting time on obvious rejects and catches failures such as stable dictionary substrings, repeated chunks, common words, and breach-like variants.

Reproducibility

```
data/stage0_bad_substrings.csv: checked-in pattern resource
scripts/generate_stage0_bad_substrings.py: provenance script
src/password_router_demo.py: Python code extracted from the final notebook
```

LLM Scoring Design

Continuation scoring with bounded token dynamic programming

Templates

```
A0_current_vulnerable:  
Very long but vulnerable password: {prefix}  
C3_task_continue_field:  
Task: continue the password prefix.\nPrefix: {prefix}\nContinuation:
```

Token DP state

The DP state tracks token histories that decode to a target character index. Different token histories are not collapsed just because they produce the same visible prefix; transformers condition on token sequences, not raw character strings. The bounded beam keeps the computation usable.

Original router policy

Rule	Value
Qwen reject	whole_score < 20
Qwen autoaccept	whole_score > 32
Escalation	borderline scores go to Mistral 7B
Input prefix	automatic prefix chars = 0

Capped variant

The capped notebook adds fallback edges at cost $C * \text{extra_chars}$, with $C = 2.5$. This is a policy experiment: random-looking prefixes and suffixes can help, but the marginal benefit per extra character is bounded. Its thresholds are reject below 15 and autoaccept above 25.

```
score[i] <= min_{0 <= j < i} score[j] + 2.5 * (i-j)
```

Tools Used and Repository Map

Files needed to reproduce the demo and experiments

Models and libraries

- Qwen/Qwen2.5-1.5B-Instruct: fast first-pass semantic scoring.
- mistralai/Mistral-7B-Instruct-v0.3: optional borderline escalation.
- Hugging Face Transformers and PyTorch: model loading and logits.
- zxcvbn and local Python utilities: Stage 0 scoring and substring checks.
- pandas/matplotlib: experiment result processing and figures.

Repository layout

```
notebooks/final_password_router_demo.ipynb: original uncapped demo
notebooks/capped_final_password_router_demo.ipynb: capped policy variant
notebooks/experiments/exp1d_token_dp/: reference token-DP experiment
notebooks/experiments/exp2_template_sensitivity/: continuation template experiment
notebooks/experiments/exp2b_direct_rating_negative_result/: direct-rating negative result
results/: selected reports, CSV summaries, and figures
src/password_router_demo.py: extracted demo code
```

AI interaction and verification

AI interaction is part of the method and evaluation: the project tests LLM prompts, token probabilities, direct ratings, and failure cases. AI also helped iterate on notebooks and presentation artifacts. The implementation was checked through interactive runs, saved reports, result CSVs, and repeated comparisons between intended and unintended DP variants.